

Khant Thu Aung/105292912

3.1P

Clock.cs

```
using System.Security.Principal;
```

```
using CounterApp;
```

```
using SplashScreenSDK;
```

```
namespace ClockApp
```

```
{
```

```
    public class Clock
```

```
    {
```

```
        private Counter _hour;
```

```
        private Counter _minute;
```

```
        private Counter _second;
```

```
        public Clock()
```

```
        {
```

```
            _hour = new Counter("Hour");
```

```
            _minute = new Counter("Minute");
```

```
            _second = new Counter("Second");
```

```
        }
```

```
        public void Tick()
```

```
        {
```

```
            IncrementSecond();
```

```
}  
  
private void IncrementSecond()  
{  
    _second.Increment();  
    if (_second.Ticks == 60)  
    {  
        _second.Reset();  
        IncrementMinute();  
    }  
}  
  
private void IncrementMinute()  
{  
    _minute.Increment();  
    if (_minute.Ticks == 60)  
    {  
        _minute.Reset();  
        IncrementHour();  
    }  
}  
  
private void IncrementHour()  
{  
    _hour.Increment();  
    if (_hour.Ticks == 13)  
    {  
        _hour.Reset();  
        _hour.Increment();  
    }  
}
```

```
    }  
}  
  
public void Reset()  
{  
    _hour.Reset();  
    _minute.Reset();  
    _second.Reset();  
}  
  
public string GetTime()  
{  
    return $"{_hour.Ticks:D2}:{_minute.Ticks:D2}:{_second.Ticks:D2}";  
}  
  
//properties  
  
public string Hour  
{  
    get  
    {  
        return _hour.Ticks.ToString("D2");  
    }  
}  
  
public string Minute  
{  
    get  
    {  
        return _minute.Ticks.ToString("D2");  
    }  
}
```

```

    }
}
public string Second
{
    get
    {
        return _second.Ticks.ToString("D2");
    }
}
}
}

```

Program.cs

```

using System;
using SplashKitSDK;

namespace ClockApp
{
    public class Program
    {
        public static void Main()
        {
            int secondTotal = 86400;

            Clock myClock = new Clock();

            for (int i = 0; i < secondTotal; i++)
            {

```

```
        myClock.Tick();

        Console.WriteLine(myClock.GetTime());

        // Thread.Sleep(1000);

    }

}

}
```

TestClock.cs

```
using NUnit.Framework.Legacy;

using ClockApp;

namespace ClockTests
{
    [TestFixture]
    public class ClockTest
    {
        private Clock _clock;

        [SetUp]
        public void Setup()
        {
            _clock = new Clock();
        }

        [Test]
        public void ClockStartsAtMidnight()
```

```
{  
    Assert.That(_clock.GetTime(), Is.EqualTo("00:00:00"));  
}
```

```
[Test]  
  
public void ClockTicksOneSecond()  
{  
    _clock.Tick();  
    Assert.That(_clock.GetTime(), Is.EqualTo("00:00:01"));  
}
```

```
[Test]  
  
public void ClockTicksSixtySeconds()  
{  
    for (int i = 0; i < 60; i++)  
    {  
        _clock.Tick();  
    }  
    Assert.That(_clock.GetTime(), Is.EqualTo("00:01:00"));  
}
```

```
[Test]  
  
public void ClockTicksOneHour()  
{  
    for (int i = 0; i < 3600; i++) // 60 * 60 = 3600 seconds  
    {
```

```

        _clock.Tick();
    }

    Assert.That(_clock.GetTime(), Is.EqualTo("01:00:00"));
}
}
}

```

TestCounter.cs

```

using NUnit.Framework.Legacy;

using CounterApp;

namespace CounterTests
{
    [TestFixture]
    public class CounterTest
    {
        private Counter _counter;

        [SetUp]
        public void Setup()
        {
            _counter = new Counter("TestCounter");
        }

        [Test]
        public void CounterStartsAtZero()
        {

```

```
#pragma warning disable NUnit2005 // Consider using Assert.That(actual,
Is.EqualTo(expected)) instead of ClassicAssert.AreEqual(expected, actual)

    ClassicAssert.AreEqual(0, _counter.Ticks, "Counter should start at 0");

#pragma warning restore NUnit2005 // Consider using Assert.That(actual,
Is.EqualTo(expected)) instead of ClassicAssert.AreEqual(expected, actual)

}
```

```
[Test]
```

```
public void CounterIncrementsByOne()
```

```
{
```

```
    _counter.Increment();
```

```
    Assert.That(_counter.Ticks, Is.EqualTo(1), "Counter should be 1 after one increment");
```

```
}
```

```
[Test]
```

```
public void CounterIncrementsMultipleTimes()
```

```
{
```

```
    for (int i = 0; i < 5; i++)
```

```
    {
```

```
        _counter.Increment();
```

```
    }
```

```
    Assert.That(_counter.Ticks, Is.EqualTo(5), "Counter should be 5 after five
increments");
```

```
}
```

```
[Test]
```

```
public void CounterResetsToZero()
```

```

{
    _counter.Increment();

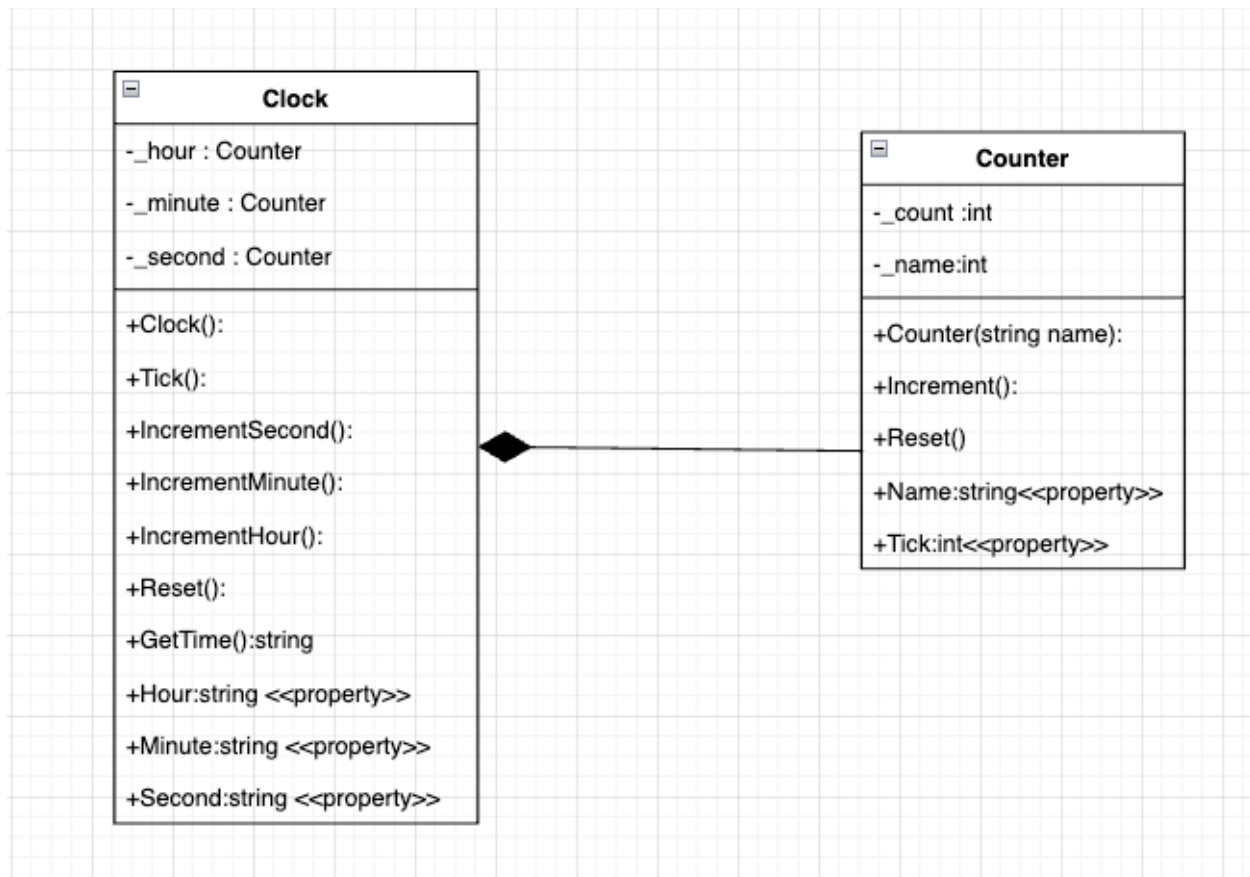
    _counter.Increment();

    _counter.Reset();

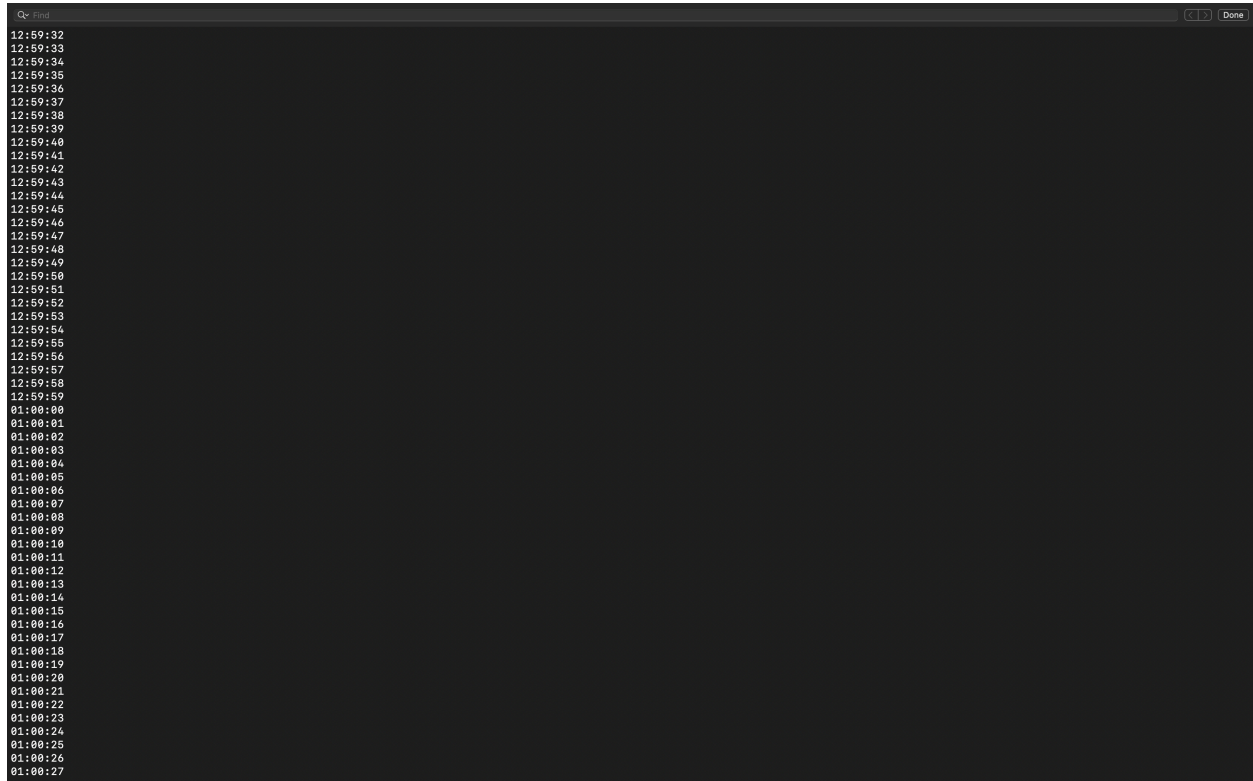
    Assert.That(_counter.Ticks, Is.EqualTo(0), "Counter should be 0 after reset");
}
}
}

```

UML diagram

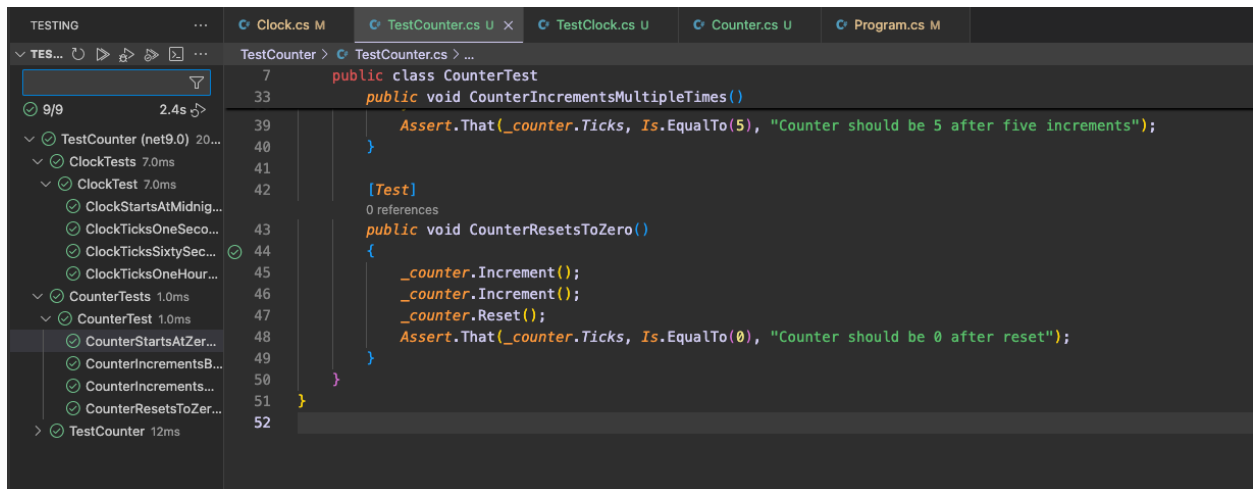


Console Output



```
12:59:32
12:59:33
12:59:34
12:59:35
12:59:36
12:59:37
12:59:38
12:59:39
12:59:40
12:59:41
12:59:42
12:59:43
12:59:44
12:59:45
12:59:46
12:59:47
12:59:48
12:59:49
12:59:50
12:59:51
12:59:52
12:59:53
12:59:54
12:59:55
12:59:56
12:59:57
12:59:58
12:59:59
01:00:00
01:00:01
01:00:02
01:00:03
01:00:04
01:00:05
01:00:06
01:00:07
01:00:08
01:00:09
01:00:10
01:00:11
01:00:12
01:00:13
01:00:14
01:00:15
01:00:16
01:00:17
01:00:18
01:00:19
01:00:20
01:00:21
01:00:22
01:00:23
01:00:24
01:00:25
01:00:26
01:00:27
```

TestExplorer



```
TESTING
...
Clock.cs M
TestCounter.cs U x
TestClock.cs U
Counter.cs U
Program.cs M

TES... 9/9 2.4s
TestCounter (net9.0) 20...
  ClockTests 7.0ms
    ClockTest 7.0ms
      ClockStartsAtMidnig...
      ClockTicksOneSeco...
      ClockTicksSixtySec...
      ClockTicksOneHour...
    CounterTests 1.0ms
      CounterTest 1.0ms
        CounterStartsAtZer...
        CounterIncrementsB...
        CounterIncrements...
        CounterResetsToZer...
      TestCounter 12ms

TestCounter > TestCounter.cs > ...
7 public class CounterTest
33 public void CounterIncrementsMultipleTimes()
39     Assert.That(_counter.Ticks, Is.EqualTo(5), "Counter should be 5 after five increments");
40 }
41
42 [Test]
43 0 references
44 public void CounterResetsToZero()
45 {
46     _counter.Increment();
47     _counter.Increment();
48     _counter.Reset();
49     Assert.That(_counter.Ticks, Is.EqualTo(0), "Counter should be 0 after reset");
50 }
51 }
52
```